SUMMER INTERSHIP REPORT

Submitted by

JARUGU GOWRI POOJITHA
(R200558)

Under the esteemed guidance of

Mrs.P.UDAYA SREE,
Assistant Professor
Department of CSE



Submitted to

Department of Computer Science and Engineering

Rajiv Gandhi University of Knowledge and Technologies
(RGUKT) R.K.Valley, Kadapa, Andhra Pradesh, 516 330
Accredited by 'NAAC' with 'B+' Grade

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RGUKT, R.K. Valley

CERTIFICATE

This is to certify that the project work titled **"Restaurant Management System"** is a bonafide project work submitted by **JARUGU GOWRI POOJITHA-R200558** in the Department of COMPUTER SCIENCE AND ENGINEERING in partial fulfillment of requirements for the Summer internship for the degree of Bachelor of Technology in **Computer Science and Engineering** for the year 2024- 2025 carried out the work under the supervision.

INTERNAL GUIDE

Mrs.P.UDAYA SREE,

Mtech,Phd,
Assistant Professor,
Dept of CSE,

RGUKT RK VALLEY.

HEAD OF THE DEPARTMENT

Dr. CH. Ratna Kumari

M.Tech,(university of hydrebad),
Ph.D(JNTU-H),
Asst. professor Dept. of. CSE,

RGUKT RK VALLEY.

Rajiv Gandhi University of Knowledge and Technologies (RGUKT), R.K. Valley, Kadapa, Andhra Pradesh, 516330

DECLARATION

I hereby declare that this project work report " Restaurant Management System"is from the company **ExcelR** under the role of a summer intern completed by **J.GOWRI POOJITHA(R200558)**from the period of may 2025 to June 2025. This work entitles as a part of summer internship Project for **Bachelor of technology in computer science and engineering** under RGUKT-RKVALLEY.

WITH SINCERE REGARDS

JARUGU GOWRI POOJITHA(R200558)

TABLE OF CONTENT

S.NO	INDEX	PAGE NO
	Acceptance Letter and Certificate of Completion	5-6
	Abstract	7
1	**Introduction**	8-11
	1.1 Background	
	1.2 Problem Statement	
	1.3 Purpose of the Project	
	1.4 Scope of the Project	
	1.5 Motivation	
	1.6 Tools and Technologies Used	
2	**Literature Survey**	12-14
	2.1 Traditional Restaurant Management Practices	
	2.2 Point of Sale and Digital Ordering Systems	
	2.3 Existing Research and Academic Projects	
	2.4 Role-Based Access Control in Restaurant Systems	
	2.5 Gaps Identified and Proposed Contribution	
3	**System Requirements**	15-18
	3.1 Functional Requirements	
	3.2 Non-Functional Requirements	
	3.3 Hardware Requirements	
	3.4 Software Requirements	
	3.5 Installation and Deployment Guidelines	
4	**User Interface and Front-End Screenshots**	19-22
	4.1 Overview	
	4.2 Login Screen	
	4.3 Staff Distribution	
	4.4 Order Creation Screen	
	4.5 Manage Employees Screen	
	4.6 Show Payment Summary Screen	
5	**Conclusion and Future Enhancements**	23-25
	5.1 Conclusion	
	5.2 Future Enhancements	
6	References	26

ACCEPTANCE LETTER:

INTERNSHIP ACCEPTANCE LETTER

Date: 08-May-2025

Dear **Gowri Poojitha Jarugu,**

We are delighted to inform you that you have been selected for the prestigious **Full Stack Java** Short Term Internship Program at ExcelR Edtech Pvt Ltd. Your commitment to learning and growth has set you apart, and we are excited to have you onboard.

As a student of **Rajiv Gandhi University Of Knowledge Technologies** this program will be a significant milestone in your academic and professional journey.

At ExcelR, we believe in fostering talent through experiential learning, and we are confident that this opportunity will empower you with the knowledge and expertise needed to thrive in your future career.

Best Wishes from Team ExcelR!

Mr. Ram Tavva,
CEO,
ExcelR Edtech Pvt. Ltd.

For any support, please connect with our Support Helpline at:
Mobile: +91 74119 55043 / +91 74119 55042 / (10:00 AM to 7:00 PM)
Email: internship.support@excelr.com

CERTIFCATION OF COMPLETION:



This project is taken from the company ExcelR, required for the completion of summer internship

Internship mode: online

Period of time: may 2025- June 2025

ABSTRACT

The **Restaurant Management System** is a robust, role-based desktop application developed to digitize and optimize the core operations of a restaurant. This project addresses the challenges faced by restaurants in managing orders, staff, menus, and payments through an intuitive and secure software interface. Designed using object-oriented principles, the system offers two distinct user roles—**Manager** and **Staff**—each with tailored access and functionality to maintain operational control and data integrity.

Upon login, the system automatically records the user's clock-in time, providing an embedded attendance and working hour tracking mechanism. **Managers** have elevated privileges, enabling them to **add, edit, or delete staff**, **manage menu items**, and **view daily payment summaries**. They are also empowered to clock out employees manually when necessary, ensuring accurate time logs even during anomalies.

Staff members are provided with tools to handle all customer-facing order operations. They can **create new orders**, **add or remove items**, **edit existing orders**, and **close or cancel** them. Menu items are categorized into **Drinks, Alcohol, Main Courses, and Desserts**, making navigation and item selection efficient. The order interface ensures accuracy by allowing quantity input, real-time updates, and immediate reflections in the order summary.

The system also incorporates a **payment tracking module**, where each closed order contributes to a daily financial record, enabling managers to assess daily revenue and performance. All user actions are logged, ensuring accountability and traceability.

From a technical perspective, the system is structured to follow the **Model-View-Controller (MVC)** pattern, supporting scalability and maintainability. The application can be implemented using **Java** for the backend with **JavaFX or Swing** for the frontend, and a **relational database** such as MySQL or SQLite for data persistence.

By automating tasks that were traditionally manual—such as staff clock-in/out, order processing, and menu updates—this Restaurant Management System significantly reduces human error, enhances efficiency, and provides a seamless operational experience. The modular design allows for future expansion, including potential integration with **online ordering systems**, **mobile applications**, and **digital payment gateways**, making it a forward-compatible solution for modern restaurants.

Chapter1: Introduction

1.1 Background

Restaurants are dynamic environments where efficiency, speed, and accuracy are critical to customer satisfaction and business success. From taking orders to generating bills, updating menus, managing employees, and handling daily transactions, these tasks require coordination between various departments and personnel.

Historically, these functions were handled manually—orders were written on paper, menus updated by reprinting hard copies, and payments recorded in ledgers. Such practices led to frequent errors, delays, employee time misuse, and lack of centralized reporting. Moreover, during peak hours, miscommunication between waiters and the kitchen often resulted in incorrect or delayed food delivery, frustrating customers and impacting restaurant credibility.

With the growth of the digital economy, automation and software-driven solutions have become not just optional but essential. Restaurants now aim to adopt digital systems that can centralize operations, reduce dependency on manual work, and provide real-time insights into sales, employee performance, and inventory status.

This project aims to design and implement a **Restaurant Management System (RMS)** that combines these capabilities into a single, easy-to-use platform—bridging the gap between traditional hospitality service and modern digital management.

1.2 Problem Statement

Despite the growing popularity of technology in retail and hospitality sectors, many small to mid-scale restaurants still rely on fragmented or manual processes. The absence of an integrated system results in several operational issues:

- **Order Errors:** Handwritten orders can be misread or misplaced, leading to wrong deliveries and wastage.
- **Menu Inflexibility:** Prices and item availability are hard to update instantly, especially when printed menus are used.
- **No Staff Accountability:** Without digital check-in systems, tracking staff attendance, punctuality, and shift durations is difficult.
- **Limited Managerial Oversight:** Managers lack real-time insights into sales, employee activities, and order status.
- **Manual Billing & Reports:** Calculating daily revenue or generating summaries takes time and is error-prone.

These limitations reduce customer satisfaction, employee efficiency, and the business's ability to make data-driven decisions.

1.3 Purpose of the Project

The purpose of this project is to develop a secure, reliable, and feature-rich **Restaurant Management System** that:

- **Automates** day-to-day operations such as order processing, menu management, billing, and staff monitoring.
- **Provides role-based access** to ensure sensitive functions like employee and menu management are restricted to managers.
- **Reduces order-processing time** through an intuitive GUI that enables quick item selection, quantity input, and order editing.
- **Enhances transparency** by logging all user activities, from login time to orders handled.
- **Tracks staff work hours** automatically upon login and logout to support performance reviews and payroll.
- **Maintains real-time payment summaries** for revenue tracking and financial planning.

The system supports local data storage but can be extended to connect to cloud databases or integrate with third-party payment systems.

1.4 Scope of the Project

This RMS application is designed as a **single-branch, desktop-based software system** to handle internal restaurant operations. The current version focuses on the following key modules:

Core Modules:

- **User Authentication System**: Secure login for managers and staff with hardcoded test credentials.
- **Menu Management**: Add, edit, delete food and drink items, organized by category (Drink, Alcohol, Main Course, Dessert).
- **Order Management**: Create and edit orders, add/remove items, close/cancel orders with quantity control.
- **Employee Management**: Manager-exclusive feature to manage staff profiles, monitor work hours, and enforce attendance.
- **Payment Summary**: Auto-generated report of daily closed orders to help track revenue.

Future Scope:

- Integration with online ordering portals
- Mobile/tablet POS application
- Kitchen display system (KDS) for real-time order viewing
- Loyalty programs and customer feedback module
- Multi-branch and cloud database support

This makes the system suitable for scalability and adaptation in both small independent restaurants and larger franchises.

1.5 Motivation

This project is motivated by real-world challenges observed in the restaurant industry, particularly in environments where staff are overburdened and operational tools are outdated or inefficient. The constant need for quicker service, fewer errors, and better transparency inspired the creation of a system that acts as a digital assistant to restaurant employees and managers.

Additionally, from a developer's perspective, this project serves as an opportunity to apply and enhance knowledge in:

- Object-oriented programming using Java
- GUI development using JavaFX/Swing
- Database connectivity using JDBC
- Data modeling and system architecture
- Software development lifecycle and real-world system design

It is also motivated by the industry's shift towards **smart hospitality solutions**, where automation and data are key to driving customer satisfaction and operational excellence.

1.6 Tools and Technologies Used

Category	Tool / Technology	Purpose
Programming Language	Java	Core backend development
GUI Framework	JavaFX / Swing	Frontend interface
Database	MySQL / SQLite	Persistent storage of users, menu, orders
Database Connector	JDBC (Java Database Connectivity)	Communication between Java app and database
IDE	IntelliJ IDEA / Eclipse	Code development and debugging
Diagramming Tools	Draw.io / Pencil / Lucidchart	UML diagrams (Use Case, Class, Sequence)
Version Control	Git	(Optional) Project version management

The stack was selected to ensure platform independence, maintainability, and adherence to industry-standard design principles.

Chapter 2: Literature Survey

2.1 Traditional Restaurant Management Practices

Historically, restaurant operations were largely manual. Orders were recorded on paper, menus were printed, and bills were calculated by hand. Staff attendance was noted using registers or verbal confirmations. This led to a variety of challenges:

- Prone to human error (incorrect orders, inaccurate bills)
- Time-consuming during rush hours
- Difficult to track employee hours and wages
- No real-time reporting or analytics

These manual practices, while simple, lacked scalability and transparency. As restaurants expanded or started handling delivery orders, the inefficiencies became more apparent. Traditional models failed to support modern needs such as digital payments, real-time updates, and remote management.

2.2 Point of Sale (POS) and Digital Ordering Systems

Modern restaurants have increasingly adopted **Point of Sale (POS)** systems that allow digital ordering, menu management, and integrated billing. POS systems often feature:

- Touchscreen terminals for order input
- Integrated billing and receipt generation
- Kitchen display system (KDS) integration
- Inventory and stock monitoring

However, many low- and mid-tier restaurants still face challenges:

- High cost of commercial POS systems
- Complex interfaces unsuitable for smaller teams
- Limited customization for region-specific workflows
- Often no modular support for staff or employee tracking

Some notable systems include **Toast POS**, **Square for Restaurants**, and **Zomato Base**. These systems are effective but might be overkill or financially unfeasible for smaller, independent

restaurants. This opens a space for lightweight, custom-built systems like the one proposed in this project.

2.3 Existing Research and Academic Projects

Several academic studies and student-led projects have addressed similar problems in restaurant automation. A few notable examples:

- **"A Java-based Restaurant POS System"** – Designed using Java and MySQL, focused on order entry and printing bills. However, it lacked modules for employee management or time tracking.
- **"Restaurant Automation System using Python & Django"** – Web-based solution integrating online orders, but the setup complexity limited its offline usability.
- **"Design and Implementation of a Mobile-Based Restaurant Ordering System"** – Explored mobile-first ordering but didn't address back-end staff or payment tracking.

Most of these solutions focus on either **customer experience** (e.g., mobile ordering) or **billing**, but few offer a **full-stack internal system** that supports both **managerial control** and **staff-side operation** with a local database.

2.4 Role-Based Access Control (RBAC) in Restaurant Systems

Role-Based Access Control (RBAC) is a well-researched model in software security where system access is limited based on user roles. In restaurant systems, this ensures:

- Only **managers** can edit menus, manage employees, or view financial reports
- **Staff** can access only the order management and clock-in/clock-out features

RBAC improves **security, accountability, and system integrity**. Studies show that RBAC significantly reduces the likelihood of unauthorized modifications and promotes better workflow delegation in software systems.

By integrating RBAC, this project ensures that each user interacts with only the features relevant to their role, minimizing misuse and simplifying the user interface.

2.5 Gaps Identified and Proposed Contribution

From the survey of existing solutions and research, the following **gaps** are identified:

- Lack of lightweight, offline-compatible systems for single-branch restaurants
- Few systems offer **employee time tracking** and **attendance logging**

- Menu management is often hard-coded or lacks proper categorization
- Limited integration between order handling and payment tracking
- Lack of open-source systems that are customizable by local developers

This project addresses these gaps by offering:

- A role-based, modular application in Java with a local database
- Automatic clock-in/out with user login
- Menu categorization and item-based quantity management
- Integrated order and payment tracking for daily reporting
- GUI-based design for ease of use without deep technical knowledge

This contribution offers a balance between the simplicity of traditional systems and the power of modern restaurant technology—making it a viable solution for smaller, cost-conscious restaurants.

Chapter 3: System Requirements

3.1 Functional Requirements

Functional requirements specify what the system should do. They define the essential features that support the restaurant's core operations.

1. User Authentication

The system must support secure login for two types of users—**Managers** and **Staff**. Each user must enter a valid ID and password to access the system. Upon successful authentication, the system should identify the user's role and present the appropriate dashboard. Managers will have access to advanced functions such as menu and employee management, while staff will be restricted to order handling and clock-in/out.

2. Menu Management

Accessible only to managers, this module allows adding, updating, and deleting menu items. Each item must have attributes such as item name, price, and category. Categories include **Drink**, **Alcohol**, **Main Course**, and **Dessert**, enabling structured organization of menu items for quick access during order-taking. The system must instantly reflect any changes made to the menu for real-time consistency.

3. Order Management

Staff should be able to create new customer orders, add items from the menu, specify quantities, and edit or delete them as needed. An open order can be edited until it is either closed or canceled. The interface must support quick item selection to facilitate faster customer service, especially during peak hours. Once closed or canceled, the order must be locked from further modifications to maintain integrity in payment logs.

4. Employee Management

This is a manager-exclusive module that enables the addition of new staff members, editing their details, or removing them when needed. The system must maintain a database of staff credentials, ensuring each employee has a unique ID and password. Managers can also view staff activity and clock them out in case they forget to do so manually, ensuring accurate time tracking.

5. Payment Tracking

The system should automatically calculate the total for each order and generate a daily payment summary accessible to the manager. Each closed order must contribute to the day's earnings. The

system should present a breakdown of daily transactions, assisting in revenue monitoring, shift reconciliation, and future decision-making.

6. Attendance and Time Logging

Every login should act as a clock-in for the user, while the “Clock Out” button should mark the end of their working session. If the user forgets to clock out, the manager should be able to perform this action on their behalf. This feature helps maintain discipline and transparency in employee attendance records, which can be useful for payroll and shift planning.

3.2 Non-Functional Requirements

Non-functional requirements describe how the system should perform under various conditions and ensure the system’s quality attributes such as reliability, scalability, and usability.

1. Usability

The system must provide an intuitive and easy-to-navigate graphical user interface (GUI). Clear labels, consistent button placement, and logical workflows are critical to reduce the learning curve for new users, especially those who may not be tech-savvy.

2. Performance

The system must be responsive, even during heavy usage, such as managing multiple simultaneous orders. It should load interfaces swiftly and respond to user actions without lag, maintaining performance even with increasing data (e.g., more menu items or employees).

3. Security

Only authenticated users must be allowed access to the system. Sensitive features like employee management and financial data must be restricted to managers only. Passwords should be stored securely, and role-based access control must be strictly enforced to prevent unauthorized use.

4. Reliability

The system should operate consistently without crashing. All data—such as orders, payments, and employee actions—must be reliably stored and retrievable in subsequent sessions. Even in the event of abrupt shutdowns, data integrity should be preserved through proper transaction handling.

5. Scalability

Although designed for a single-branch setup, the system architecture should allow future upgrades, such as transitioning to a client-server model or integrating cloud-based services.

6. Maintainability

The system should have a modular structure, where individual components like order management or menu management can be debugged or upgraded independently. This makes future enhancements and maintenance cost-effective and simple.

7. Portability

Since the system is built using Java, it must be platform-independent and executable on any system that supports the Java Virtual Machine (JVM), including Windows, Linux, and macOS.

3.3 Hardware Requirements

To ensure smooth execution, the system requires basic hardware components commonly available in most restaurant environments. A standard desktop or laptop is sufficient.

The application should run comfortably on a machine with a **minimum Intel i3 or equivalent processor**, **4GB of RAM**, and around **500MB of free disk space**. The system should support standard peripherals such as a **mouse and keyboard** for navigation, and the display should be at least **1024x768 pixels** to ensure GUI elements are properly visible.

No specialized hardware is required, making this system viable for deployment even in budget-constrained settings.

3.4 Software Requirements

The system requires a standard Java development environment. The backend logic is written in Java, utilizing either **JavaFX or Swing** for the frontend. For persistent storage, it uses **MySQL** or **SQLite** as the database system, and **JDBC** is used to connect Java applications to the database.

The development can be carried out in any modern IDE, such as **IntelliJ IDEA**, **Eclipse**, or **NetBeans**. The database management interface can be accessed through **MySQL Workbench**, **DBeaver**, or the command-line interface.

To draw and manage system design diagrams, tools like **Draw.io**, **Lucidchart**, or **Pencil Project** can be used.

3.5 Installation and Deployment Guidelines

To install and deploy the Restaurant Management System:

1. Install the **Java Development Kit (JDK)** on the machine.
2. Install a **Java-compatible IDE** (IntelliJ/Eclipse).

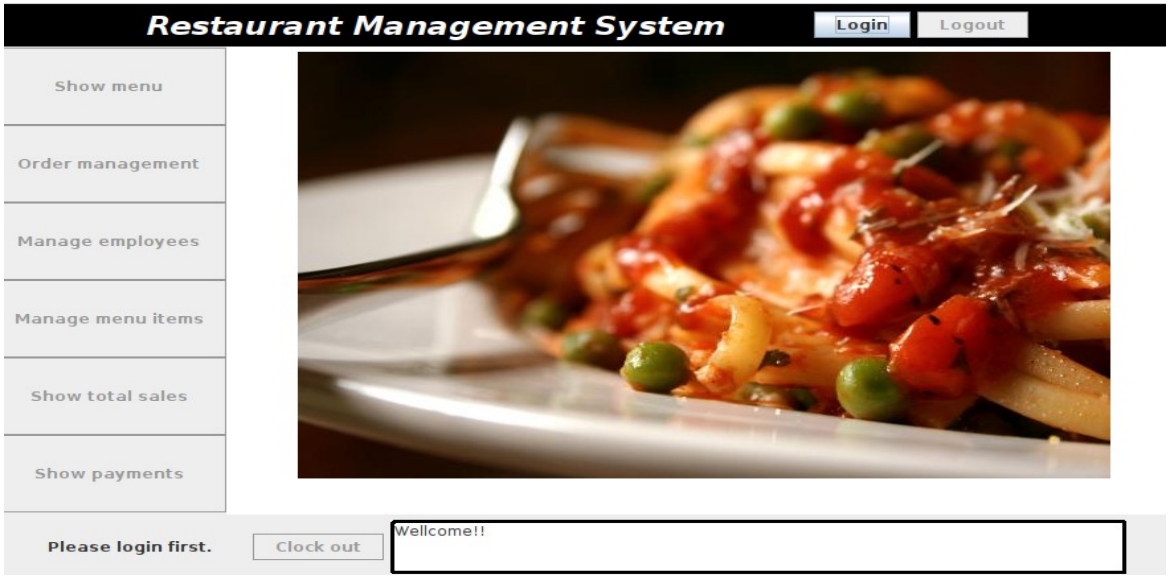
3. Set up **MySQL or SQLite**, depending on the preference for lightweight or more scalable database solutions.
4. Clone or extract the project source files into the workspace.
5. Configure the JDBC connection with proper database credentials.
6. Compile and run the project using the IDE.
7. Generate a runnable .jar file for use in production environments.

After successful deployment, restaurant staff can log in with predefined credentials and begin operations. Managers can initialize the menu and employee data during the first-time setup.

Chapter 4: User Interface and Frontend Screenshots

4.1 Overview

The success of any management system heavily relies on how intuitive and user-friendly its interface is. The Restaurant Management System is built using JavaFX (or Swing), offering a clean and responsive graphical user interface (GUI). This section includes key frontend screenshots and explains the purpose and design of each screen.

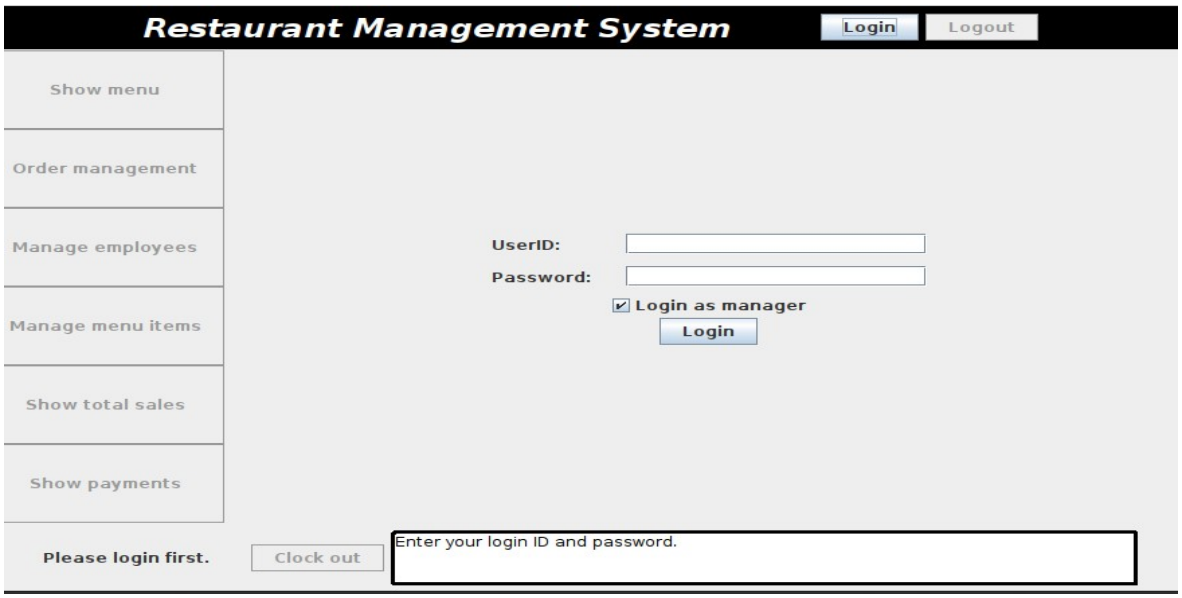


4.2 Login Screen

Description:

The login screen is the entry point to the system. It requires the user to enter a valid **User ID** and **Password**. Based on their credentials, users are authenticated as either **Manager** or **Staff**.

- If credentials are correct: Redirected to respective dashboard
- If credentials are incorrect: An error message is displayed
- On successful login: The system records the **clock-in time**



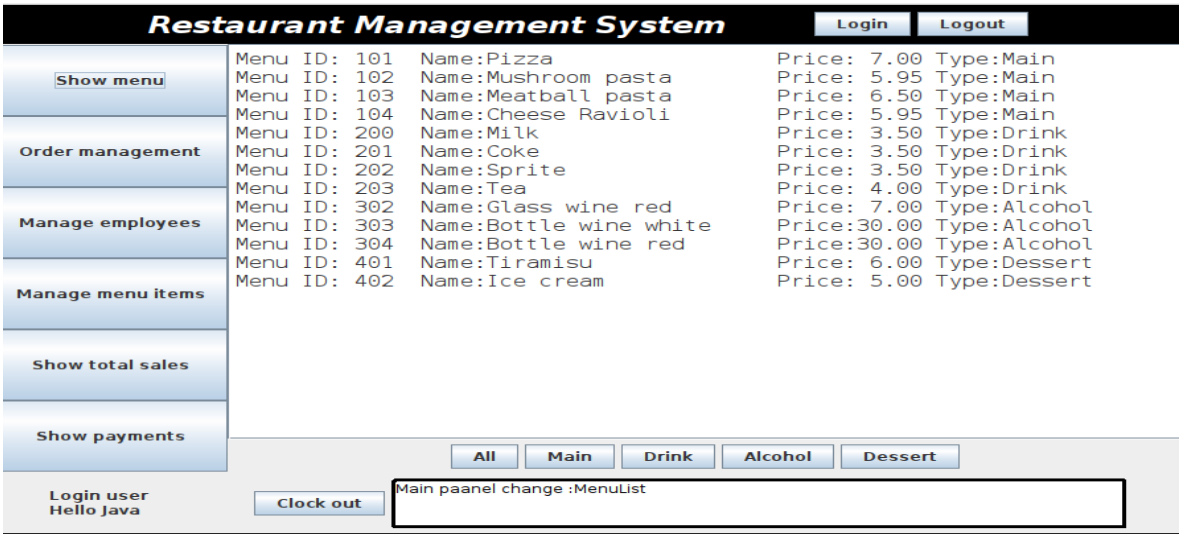
4.3 Staff Dashboard

Description:

Once a staff member logs in, they are presented with the order management interface. The left panel contains buttons to:

- Show the menu
- Create a new order
- Clock out

The right panel lists available **menu items**, grouped by categories (Drink, Alcohol, Main, Dessert), each dynamically clickable. The center panel displays the **order summary**.



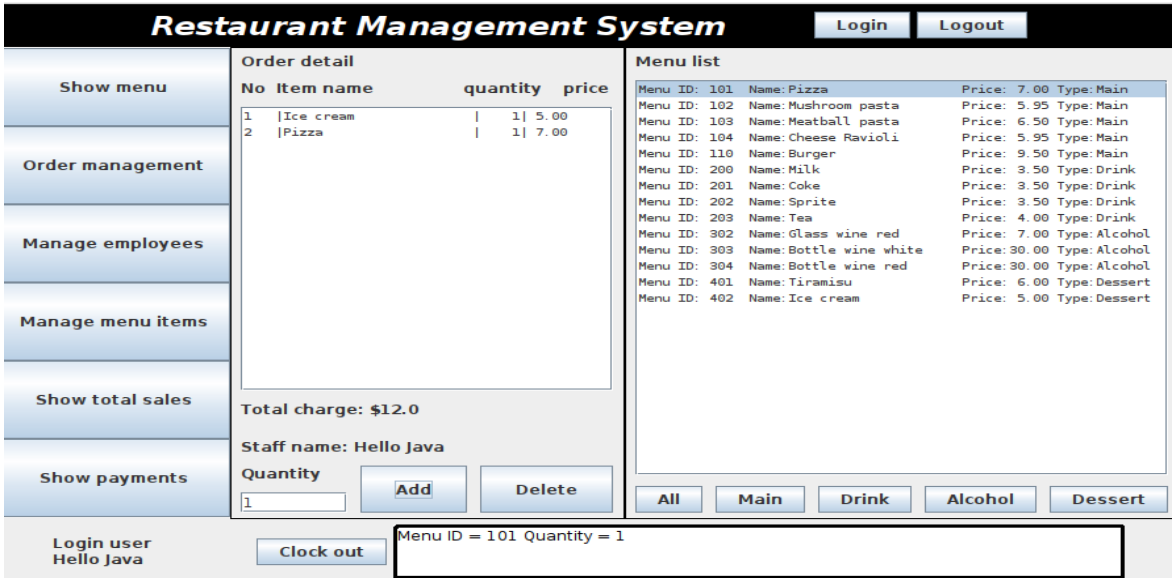
4.4 Order Creation Screen

Description:

Here, staff can:

- Click on menu items to select them
- Enter quantity for each item
- Click **"Add"** to add items to the order list
- See running totals and itemized order summary
- Use **Delete** to remove mistakenly added items
- Use **Edit/Close/Cancel** to modify or finalize the order

This screen is optimized for quick service during busy hours with minimal input and immediate feedback.



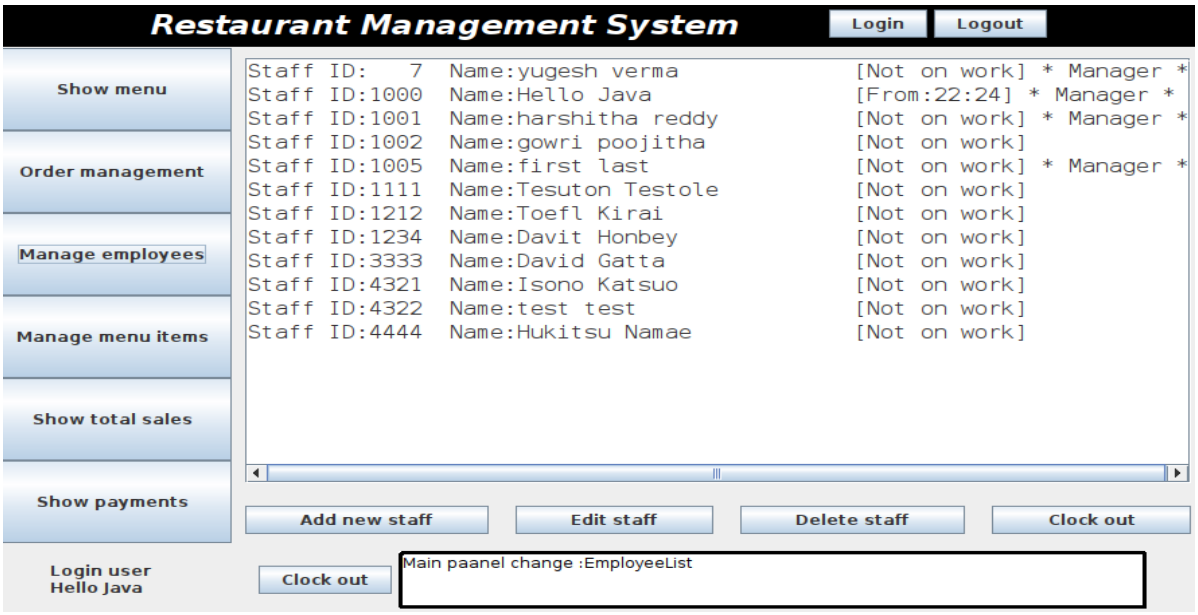
4.5 Manage Employees Screen

Description:

The manager can:

- Add new staff with user ID, name, and password
- Edit existing staff details
- Delete staff from the database
- Manually clock-out any logged-in employee

The employee list is displayed with filtering or selection options to ease navigation.

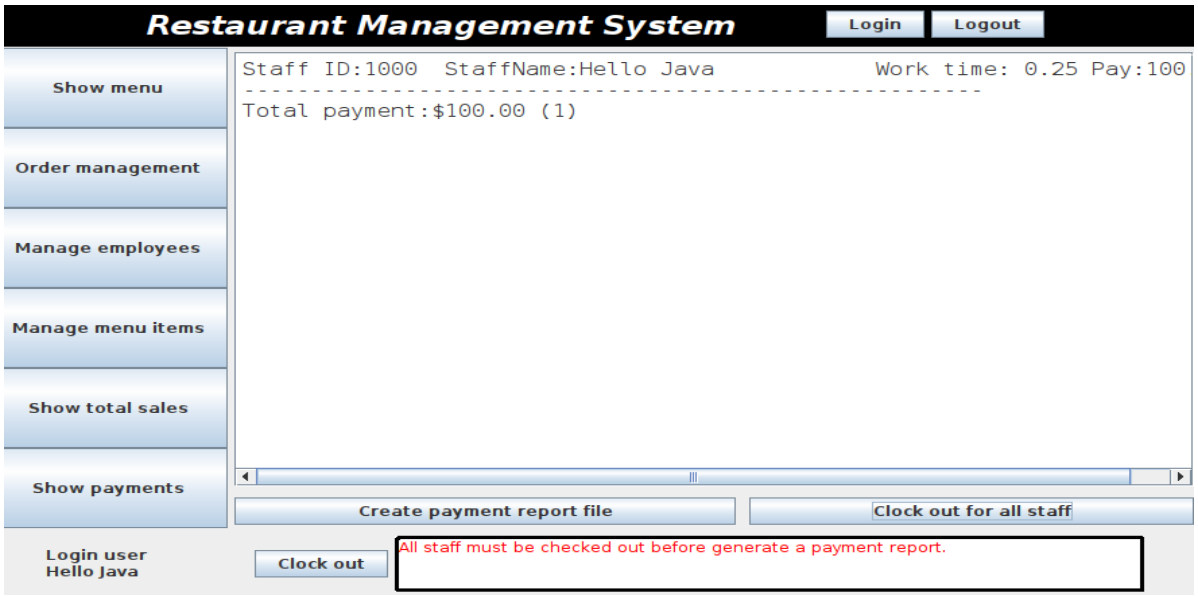


4.6 Show Payment Summary Screen

Description:

Displays a list of all **closed orders** for the current day:

- Each entry includes order ID, total amount, and timestamp
- Helps in auditing daily earnings and shift performance
- Essential for financial reconciliation at the end of day



Chapter 5: Conclusion and Future Enhancements

5.1 Conclusion

The development of the **Restaurant Management System (RMS)** has successfully addressed the core challenges faced by traditional and semi-digital restaurant environments. This project provides a centralized, role-based solution that automates and simplifies various operations such as order processing, menu management, staff attendance tracking, and daily payment reporting.

By implementing this RMS using Java and a lightweight database (like MySQL or SQLite), the system ensures platform independence, offline capability, and flexibility for local customization. Through an intuitive graphical user interface (GUI), the system ensures that both technical and non-technical users can navigate and operate it efficiently.

Key accomplishments of this project include:

- **Secure login system** with role-based access control.
- **Real-time order management** with options to create, update, close, and cancel orders.
- **Menu management** with dynamic categorization of food and drink items.
- **Employee management** with clock-in/clock-out tracking and manual override capabilities.
- **Daily payment summaries** for quick revenue analysis and shift audits.

The design emphasizes modularity, scalability, and maintainability, allowing the system to serve as a practical solution for small and medium-sized restaurants that cannot afford complex, expensive POS systems. In addition to improving customer service speed and reducing manual errors, the RMS also supports administrative transparency and operational efficiency.

Overall, this project combines theoretical knowledge of object-oriented programming, user interface design, and database integration to solve a real-world problem through a full-stack application. It also reflects how even basic digital solutions can create significant value in fast-paced service industries.

5.2 Future Enhancements

While the current system fulfills essential restaurant functions, there is significant scope for further development to meet broader business needs and accommodate technological trends. Some of the proposed enhancements are listed below:

1.Web and Mobile Platform Support

Transitioning the desktop-based application to a **web-based** or **mobile application** (using technologies like ReactJS, Flutter, or Android SDK) would enable:

- Remote access and control by managers
- Customers placing orders through smartphones or QR codes
- Real-time notifications for kitchen and delivery staff

2.Cloud-Based Storage and Multi-Branch Support

Migrating the database to a **cloud-hosted solution** (e.g., Firebase, AWS RDS) would allow:

- Synchronization of data across multiple branches
- Centralized reporting for chain restaurants
- Daily backups and improved data recovery

3.Inventory and Stock Management

Adding inventory management would help:

- Track stock levels of ingredients or beverages
- Notify when items run low
- Prevent the sale of out-of-stock items

This would tie directly into the menu system and assist in procurement planning.

4.Customer Relationship Management (CRM)

Integrating CRM features would allow:

- Capturing customer details and preferences
- Managing loyalty programs and promotional offers
- Gathering feedback and analyzing service satisfaction

Such features could enhance customer retention and personalized marketing.

5.Integration with Payment Gateways and Printers

Allowing direct payment processing using services like Razorpay, Stripe, or Paytm would:

- Enable card/UPI payments from within the system

- Automatically generate digital receipts

Additionally, integrating with thermal printers or kitchen displays would speed up service.

6.Analytics and Reporting Dashboard

Developing advanced analytics modules would offer:

- Graphical dashboards for sales, staff activity, and performance trends
- Exportable reports in formats like Excel or PDF
- Insights to assist in business decision-making

7.Multilingual and Accessibility Support

Adding support for multiple languages and accessibility features (like keyboard shortcuts, screen reader support) would make the system more inclusive for diverse users and regions.

References

1. **Ian Sommerville**, *Software Engineering*, 10th Edition, Pearson Education, 2015.
   → A foundational reference for software development processes, requirement analysis, and system design principles used throughout the project.
2. **Grady Booch, James Rumbaugh, Ivar Jacobson**, *The Unified Modeling Language User Guide*, 2nd Edition, Addison-Wesley, 2005.
   → Used for understanding and constructing UML diagrams like class, use case, and sequence diagrams.
3. **Oracle Java Documentation** – https://docs.oracle.com/javase/
   → Official documentation for Java APIs, JavaFX components, and JDBC used in the backend development of the system.
4. **MySQL Documentation** – https://dev.mysql.com/doc/
   → Reference for SQL syntax, database design, and integration of MySQL with Java via JDBC.
5. **JavaFX Tutorial – Oracle and OpenJFX** – https://openjfx.io/
   → Guide for building modern GUI applications with JavaFX used in designing the front-end interface of the RMS.
6. **Toast POS Features Overview** – https://pos.toasttab.com/
   → Used as a benchmark to compare common features in modern POS systems and identify gaps for this lightweight RMS.
7. **Square for Restaurants** – https://squareup.com/us/en/point-of-sale/restaurants
   → Referenced for understanding industry-standard practices in restaurant billing, payment tracking, and menu updates.
8. **A Java-Based POS System Design for Restaurants**, International Journal of Computer Applications (IJCA), Volume 176, No. 17, April 2020.
   → Academic paper that inspired the use of Java and MySQL for local restaurant operations.
9. **Design and Implementation of a Restaurant Management System using Python and Django**, International Journal of Computer Science and Information Technologies (IJCSIT), Vol. 6 (3), 2015.
   → Provided insights into full-stack design of restaurant software and its limitations in web-only contexts.
10. **Draw.io UML Diagram Tool** – https://www.diagrams.net/
    → Used for creating and exporting the UML diagrams (use case, class, and sequence diagrams) used in this project.